Modeling Human Activity States Using Hidden Markov Models

Hassan Adelani Luqman | Machine Learning Techniques 2

Formative 2 — Hidden Markov Models • 31 October 2026

Repository: https://github.com/Hassan-Adelani-Luqman/hmm-elderly-activity-monitoring

Background and Motivation

Falls and long, unnoticed periods of inactivity are serious risks for older adults who live alone, and a phone they already carry could watch for both. My use case is a monitor that recognises everyday activity states from the phone's accelerometer and gyroscope and flags unusual patterns, such as a sudden impact followed by a long stillness, for a carer to check. The activity is never measured directly, only noisy sensor signals are, so I model it as a hidden state sequence: a Hidden Markov Model over four states (still, standing, walking and jumping). Jumping stands in for a sudden impact, because real falls cannot be recorded safely.

Data Collection and Pre-processing

I recorded all data myself with Sensor Logger 1.68 on an Infinix X6855 (Android 36): 52 single-activity clips, 13 per activity, indoors on 25, 27 and 28 October 2026. For standing, walking and jumping I held the phone at waist level, screen facing me and top edge up; for still, the phone lay flat on a table. Each clip was cropped to its steady part (5–10 s), giving at least 90 s per activity (Table 1). I used TotalAcceleration (m/s², with gravity) rather than the gravity-free stream, because gravity's direction shows how the phone is held.

Table 1. Collection summary and sampling rates (single participant, one phone).

Recordings	Files / retained seconds	App setting → measured → harmonised
Activity clips	52; still 130.00, standing 119.84, walking 112.72, jumping 99.92 s	100 → 58.44–58.46 → 25 Hz
Lower-rate pilot	1 recording (phone flat)	50 → 29.23 → 25 Hz

Data-collection videos: Still | Standing | Walking | Jumping

Fig. 1 — Raw sensor magnitudes

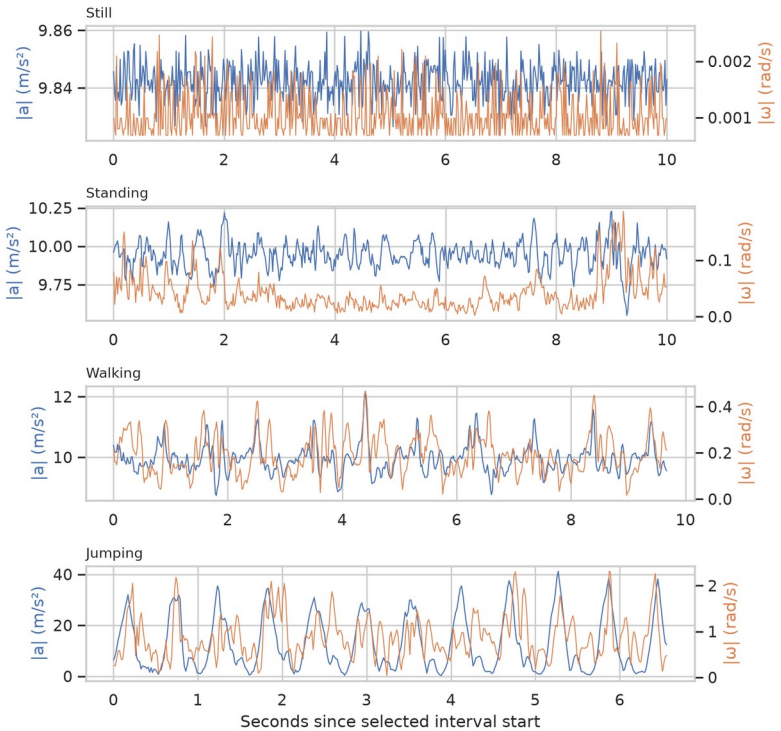


Figure 1. Raw |a| (blue, left axis) and |ω| (orange, right axis) for one clip per activity, before filtering. Axis ranges differ by activity: still varies by about 0.02 m/s², standing shows small sway, walking a regular step rhythm, and jumping large periodic impacts near 40 m/s².

Sampling rates, cleaning and windows

A pilot recording showed that the phone does not deliver the rate set in the app: the 100 Hz setting gives 58.4 Hz and the 50 Hz setting gives 29.2 Hz. The accelerometer and gyroscope are also sampled at different instants and never share a timestamp. I therefore harmonise every recording the same way: a fourth-order zero-phase Butterworth low-pass filter at 10 Hz, then one linear interpolation of both streams onto a common 25 Hz grid. I chose 25 Hz because it is below every delivered rate, so no recording is ever upsampled, and its 12.5 Hz Nyquist limit still covers the walking and jumping rhythms (about 1.5–2.5 Hz) and their main harmonics.

My two planned continuous sessions (about two minutes each, covering all four activities) suffered sensor outages, including an 87.5 s gap in session 2, and I had no exact record of their activity boundaries. I did not fill or bridge the gaps: I recovered four unbroken blocks without labels, kept them out of training, and use them only to inspect decoded transitions (Figure 4).

Windows are 2 s long (50 samples at 25 Hz) with a 1 s hop. Two seconds holds three or more step or jump cycles and gives 0.5 Hz FFT resolution; because the window is defined in seconds, it covers the same time span whatever a recording's native rate. Each window takes the majority label of its samples, and windows never cross file boundaries, giving 338 training and 65 test windows.

Table 2. The ten retained features and why each helps separate the activities.

Features	Domain	Justification
Mean acc_x; mean acc_y	Time (2)	Gravity direction: flat phone (still) versus upright phone (other activities)
Std of |a|	Time	Motion intensity: still < standing < walking < jumping
Variance of |ω|	Time	How much the phone rotates while moving
SMA acceleration; SMA gyro	Time (2)	Overall activity level and rotation (mean sum of absolute axes)
Correlation acc_y, acc_z	Time	Coordinated movement between axes versus uncorrelated noise
Dominant frequency of |a|	FFT	Step or jump rate; no clear peak when still or standing
Band-energy ratio of |a|	FFT	Low-band gait energy [0.5,3] Hz versus higher band (3,10] Hz
Spectral entropy of |a|	FFT	Periodic motion has low entropy; sensor noise has high entropy

I computed 14 candidate features and dropped four (mean acc_z, centred RMS, range and spectral energy) that correlated above $|r| = 0.95$ with a retained feature in the training data, leaving 7 time-domain and 3 FFT features. FFT features use the demeaned |a|, a Hann window and one-sided power, ignoring the DC bin. All features are Z-score normalised with means and standard deviations from the 44 training clips only. Z-scores suit Gaussian emissions: the raw features mix m/s², rad/s, Hz and unitless ratios, and without scaling the largest-unit features would dominate every likelihood. Fitting on training data only keeps test information out of the model.

HMM Setup and Implementation Details

Table 3. HMM components (implemented from scratch in NumPy).

Element	Definition
Z	Four hidden states: still, standing, walking, jumping
X	One ten-dimensional, Z-scored feature vector per 2 s window
A	4×4 transition matrix: A[i,j] = P(next state j | current state i)
B	Gaussian emission density per state, diagonal covariance, variance floor 1e-3
π	Initial-state probabilities; one sequence start per recording

Three algorithms are needed. The forward–backward algorithm, in scaled log space to avoid underflow, gives the sequence likelihood and the state posteriors. Baum–Welch (EM) uses those posteriors to re-estimate the parameters and stops when the total log-likelihood improves by less than 1e-4; 500 iterations is only a safety cap. Viterbi then finds the single most probable state path. Sensitivity, specificity and accuracy are computed from one hard label per window, so Viterbi's joint path is the decoder that matches the metrics, and it uses A to keep neighbouring labels consistent.

Training starts from supervised estimates: per-activity Gaussian means and variances, and transition counts with one Laplace pseudocount per entry. Every training clip contains one activity, so the counts contain no switches. In my first version, free Baum–Welch therefore drove every unobserved transition to zero and made still and jumping absorbing states. My primary model keeps the supervised transition prior fixed and lets EM re-estimate π and the emissions; the free-A model is kept for comparison. I verified the code against exact ground truth: enumerating all 64 paths of a two-state example reproduces the likelihood, Viterbi path, posteriors and one EM update, and a known three-state model is recovered from synthetic data in 7 iterations. All 65 regression tests pass.

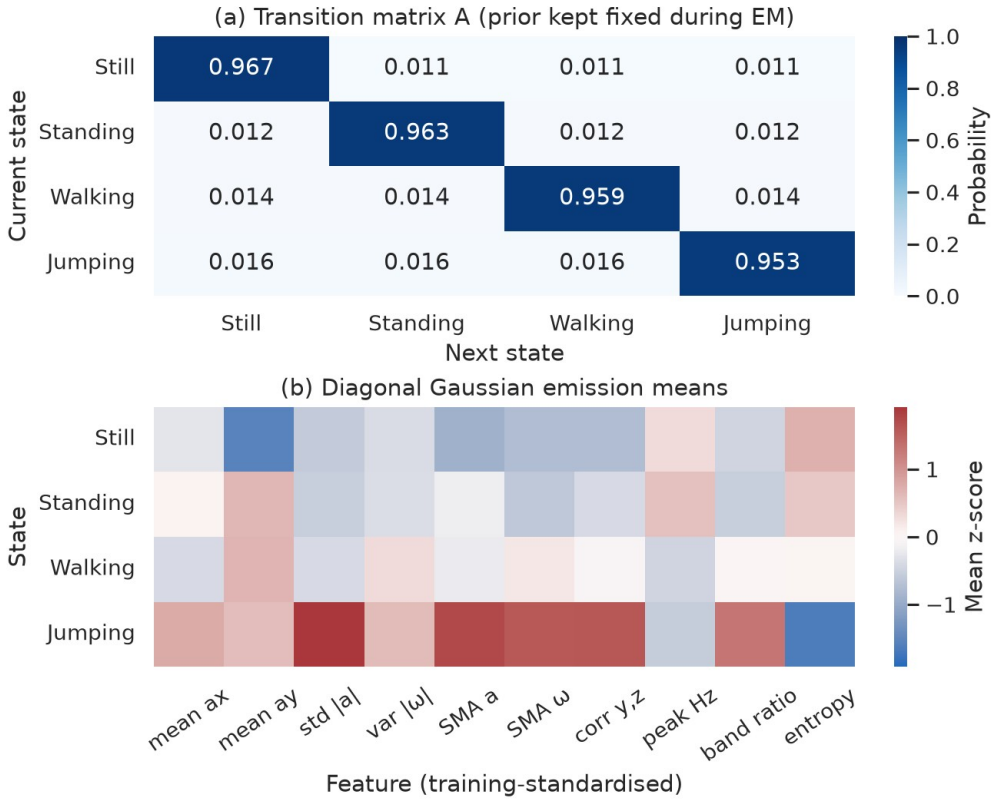


Figure 2. Primary model parameters. (a) Transition matrix A: the supervised persistence prior, kept fixed during EM; rows sum to one. (b) Gaussian emission means in training z-scores; each state also has a diagonal variance.

Each activity stays the same from one 1 s step to the next with probability 0.953–0.967, with a small equal chance of switching to any other activity. That captures one realistic property, that activities last many seconds, but not which switches are realistic: still→jumping should be far rarer than still→standing, and learning that needs labelled continuous recordings. The emission means show why the classes separate. Mean acc_y (orientation) puts still far from the three upright activities, and every motion-intensity feature puts jumping far from the rest. Standing and walking differ mainly in small motion features, which is where the errors occur.

Results and Interpretation

Before fitting anything, I held out two whole clips per activity (8 clips, 65 windows), chosen with a fixed random seed, and never used them for fitting, feature selection, scaling or state mapping. They come from the same days and conditions as training, so they test recognition on new recordings rather than in a new environment. Each test clip is decoded on its own.

Table 4. Model comparison (training accuracy is measured on the training data itself).

Model	Training correct / windows	Test correct / windows
Supervised HMM	337/338 (99.70%)	64/65 (98.46%)
Baum–Welch, A fixed (primary)	336/338 (99.41%)	63/65 (96.92%)
Baum–Welch, free A	336/338 (99.41%)	63/65 (96.92%)
Independent Gaussian (no A)	336/338 (99.41%)	64/65 (98.46%)

Table 5. Primary model on the held-out data; samples are windows.

State (Activity)	Number of Samples	Sensitivity	Specificity	Overall Accuracy*
Still	18	100.00%	100.00%	100.00%
Standing	18	88.89%	100.00%	96.92%
Walking	18	100.00%	95.74%	96.92%
Jumping	11	100.00%	100.00%	100.00%
Overall	65	97.22%	98.94%	96.92%

*Sensitivity = TP/(TP+FN); specificity = TN/(TN+FP). Activity rows give one-vs-rest accuracy (TP+TN)/65, which true negatives inflate; the Overall row gives macro sensitivity and specificity and the true multiclass accuracy (63/65).

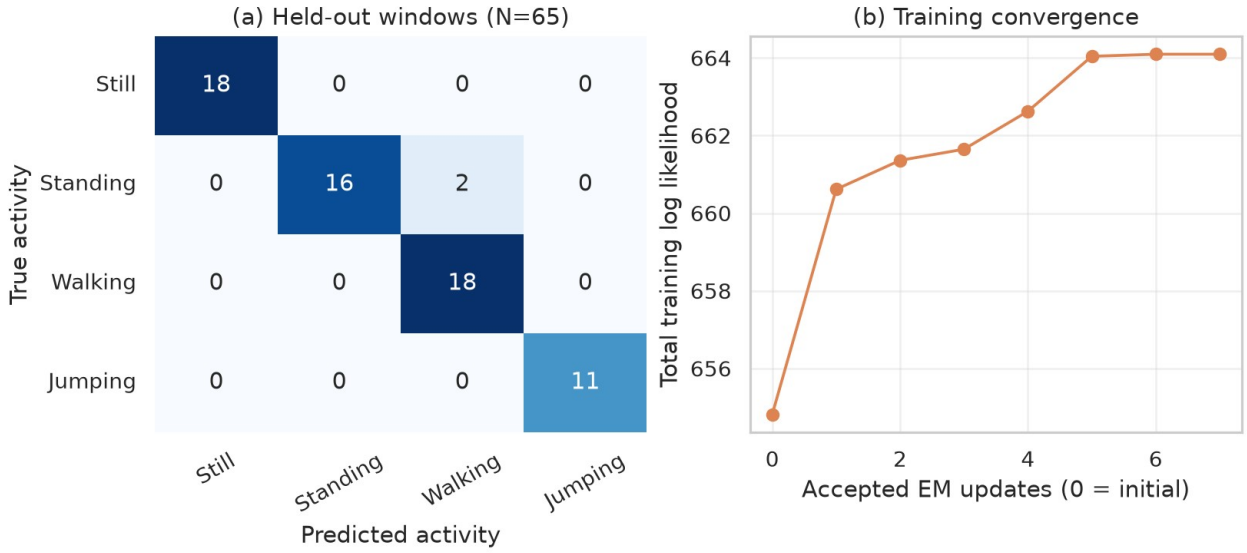


Figure 3. (a) Held-out confusion matrix for the primary model: two standing windows are predicted as walking. (b) Training log-likelihood during Baum–Welch, which stops once the improvement falls below 1e-4.

Baum–Welch raised the training log-likelihood from 654.82 to 664.10 in 7 iterations, yet both EM models classify 63/65 held-out windows (96.92%), one fewer than the supervised HMM and the baseline (98.46%). A better fit to the training data did not mean better classification. On single-activity clips the transition matrix has nothing to add, so fixing A changed no held-out prediction.

Table 6. Primary model results for each of the eight held-out clips.

Recording	Windows	Correct	Errors	Accuracy
still_02	9	9	0	100.00%
still_11	9	9	0	100.00%
standing_06	9	7	2	77.78%
standing_13	9	9	0	100.00%
walking_02	9	9	0	100.00%
walking_10	9	9	0	100.00%
jumping_02	5	5	0	100.00%
jumping_07	6	6	0	100.00%

Both errors are in standing_06 (windows 3 and 8). Window 3 coincides with a brief burst of movement in the signal, and window 8 is the clip's last window, with no later evidence to pull the decoder back to standing. The other seven clips are decoded without error, so standing sensitivity is 88.89% and walking specificity 95.74%, while still and jumping are perfect. With only two test clips per activity and overlapping windows, these figures are encouraging but not precise.

The independent Gaussian baseline classifies each window from the supervised emissions alone, ignoring A and neighbouring windows. It matches the supervised HMM here because single-activity clips leave nothing for the transitions to correct; the value of A only shows on continuous data (Figure 4). Re-running the notebook reproduces every fitted parameter and result exactly.



Figure 4. Viterbi decoding of the unlabelled continuous session 1 (three blocks; hatched gaps are sensor outages, never joined). The planned schedule is my approximate timer protocol, not verified truth. Bottom: processed |a|.

Discussion and Conclusion

Figure 4 is the most revealing result. With free transitions, the model labels the first two blocks entirely as jumping: once in an absorbing state it can never leave, so it would be useless as a continuous monitor. The primary model does switch. It finds the still periods at the start and end and the real jumping burst, but it decodes most of the session's walking and standing as jumping, because I held the phone differently in that session. Its mean acc_y sits near −0.6 z, against +0.65 z for the standing and walking clips, and the broad jumping emission absorbs these unfamiliar windows. The model generalises well to new clips recorded the same way, but not to a different grip.

Still and jumping were the easiest activities: orientation alone separates still, and jumping's impacts dominate every intensity feature. Standing versus walking was the hardest, because with the phone held steady at the waist, walking adds only a little extra motion. The pair I expected to be hardest, still versus standing, was solved by the direction of gravity.

The delivered sampling rate (58.4 Hz) differed from the app setting (100 Hz), and the two sensors never shared a timestamp, so harmonisation was essential rather than optional. Filtering at 10 Hz removes noise and aliasing but also rounds off jump impacts; that did not hurt jumping here, but it would matter for short, sharp falls. The session outages were my largest data problem and cost me labelled transition data.

The biggest improvements would be labelled continuous recordings, to learn realistic transitions and test real switching; a fixed, realistic phone placement (pocket or belt) or orientation-invariant features, so that a change of grip does not break the model; more participants and devices; and extra sensors such as a barometer for falls. A duration-aware model (a hidden semi-Markov model) would also suit activities that last many seconds.

In conclusion, my from-scratch HMM is verified against exact results and recognises held-out single-activity clips well (96.92%, with still and jumping perfect). Building it taught me that the weak points of activity recognition lie in the data rather than the algorithm: the transition matrix is only as good as the transition data, and features tied to phone orientation make the model sensitive to how the phone is held.